535510 RL Team Project Proposal: Learning Curve-Informed Reinforcement Learning for Efficient Hyperparameter Optimization

Han-Syuan Liao Cheng-An Tsai Yi-Xiang Yu
Department of Computer Science
National Yang Ming Chiao Tung University
{hans.cs12, chengan81.ii14, seann.cs12}@nycu.edu.tw

1 Project Overview

1.1 Profile

Track: 3. Application

Abstract and project goal: Hyperparameter optimization (HPO) is a fundamental yet costly challenge in machine learning: finding the right hyperparameter configuration often requires evaluating dozens of candidate configurations, each demanding a full or partial training run. Reinforcement learning (RL)-based HPO methods, such as Hyp-RL, have shown promise by learning a sequential decision policy over the hyperparameter space. However, existing RL-based approaches treat each configuration evaluation as a black-box query, discarding the rich temporal structure embedded in the learning curve produced during training.

In this project, we propose a learning curve-informed RL framework for HPO and explore two directions. (i) Derivative-informed state representation: We augment the agent's input at each step with explicit first- and second-order derivatives of the observed learning curve, providing structured inductive bias about whether a configuration is still improving, decelerating, or plateauing, at zero additional evaluation cost. (ii) Cross-dataset policy transfer: We investigate whether pre-training the LSTM policy across multiple datasets can yield an agent that generalizes to unseen datasets without restarting from scratch.

TL;DR ("Too Long; Didn't Read"): We augment an RL-based HPO agent with learning curve derivatives as structured state features, and investigate cross-dataset transfer of the learned policy for more efficient and generalizable hyperparameter search.

1.2 Motivation

- **Why is the problem interesting?**
  When human experts tune hyperparameters, they rely on the geometry of the learning curve—observing how fast the loss is dropping or whether it is plateauing—rather than treating each evaluation as an isolated scalar. We aim to provide exactly this information to an RL agent and examine whether it improves tuning efficiency. Beyond single-task HPO, a practically useful agent should also transfer its experience to new datasets, exploiting the structural regularity that learning curves share across tasks.
- **Critical challenges.**
  Challenge 1: Reliable derivative estimation from noisy learning curves. Learning curves in practice are noisy, and finite-difference estimates can amplify this noise, potentially misleading the agent. Key design questions include window size, smoothing strategy, and sensitivity of the learned policy to these choices—all of which require empirical ablation.

Challenge 2: Learning a transferable HPO policy across datasets. A practically useful HPO agent should generalize across tasks—given experience on a collection of datasets, it should be able to search effectively on a new, unseen dataset without retraining from scratch. This is non-trivial because different datasets induce different reward scales, convergence speeds, and hyperparameter sensitivities, making it difficult to learn representations that remain meaningful across tasks.

- **Justify why the problem remains open or unsolved.**
  We hypothesize that these derivative features provide a universal view of training dynamics. Since their physical meanings don't change between tasks, they make it easier for the RL agent to transfer its knowledge to new problems. Existing RL-based HPO works observe only scalar rewards, ignoring within-curve structure [Jomaa et al., 2019]. Learning curve-aware methods operate outside the RL framework [Wistuba et al., 2022]. Meta-RL approaches for HPO address cross-task generalization but rely on complex encoders without exploiting derivative structure [Albrechts et al., 2025, Wu et al., 2023]. Our work is the first to combine derivative-informed state representation with cross-dataset transfer in a unified RL-for-HPO framework.
- **State-of-the-art methods.**
  We consider Hyp-RL [Jomaa et al., 2019] as the state-of-the-art RL-based HPO baseline that our method directly builds upon.

2 Problem Formulation

In this work, we formulate the sequential hyperparameter optimization problem as a Markov Decision Process (MDP). Let Λ denote the H-dimensional hyperparameter search space. The MDP is defined by the tuple $\mathcal{M} \coloneqq (\mathcal{S}, \mathcal{A}, P, R, \gamma)$. The action space is defined as $\mathcal{A} \coloneqq \Lambda$, where an action $a_t \in \mathcal{A}$ corresponds to selecting a specific hyperparameter configuration λ_t to evaluate.

The state space $\mathcal{S}$ is defined as $\mathcal{S} \coloneqq \mathbb{R}^w \times (\Lambda \times R \times \mathcal{C} \times \mathcal{C})^*$, where each state $s_t \in \mathcal{S}$ encapsulates both the static dataset meta-features $\mathbf{d} \in \mathbb{R}^w$ and the dynamic history of previously evaluated configurations alongside their corresponding outcomes, including $\mathcal{C} \subset \mathbb{R}^K$ being the space of learning curves, representing a sequence of validation losses over K training epochs and the last two elements are the first and second derivatives of the learning curve.

The reward function is defined as $R(D, a) = -f(D, \lambda = a)$ on a given dataset D. The deterministic transition function P updates the state by appending the newly selected configuration and observed reward to the historical sequence, yielding $s_{t+1} = (s_t, (a_t, r_t, c'_t, c''_t))$. Furthermore, we impose strict termination constraints on the environment: an episode concludes either when a predefined budget sequence length T is exhausted, or if the agent selects the identical action consecutively ($a_t = a_{t-1}$).

Our objective is to learn an optimal policy $\pi^* : \mathcal{S} \to \mathcal{A}$ that maximizes the expected discounted cumulative reward, effectively identifying the hyperparameter configuration that minimizes the generalization error across varying datasets. This is achieved by estimating the optimal action-value function $Q^*(s, a)$, which satisfies the Bellman equation:

$$Q^*(s, a) = \mathbb{E}_\pi \left[r + \gamma \max_{a'} Q^*(s', a') | S_0 = s, A_0 = a \right]$$

where $\gamma \in [0, 1]$ is the discount factor. To operate within this high-dimensional state space, which consists of variable-length temporal sequences and distinct response surfaces, we employ a parameterized function approximator $Q(s, a; \theta)$. Specifically, we utilize a Long Short-Term Memory (LSTM) network to model the dynamic action-value space, enabling the agent to capture long-range dependencies and transfer optimization knowledge across datasets.

3 Empirical Evaluation

3.1 Performance Metrics

We plan to evaluate the algorithms based on the following four metrics:

- Final test/validation performance: We record the ultimate performance of the deep learning model trained with the hyperparameters discovered by the RL agent. For our continuous

regression tasks, this is measured primarily by Mean Squared Error (MSE) or Root Mean Squared Error (RMSE) on the test set.

- Search efficiency: This metric evaluates how quickly the RL agent converges to optimal hyperparameter configurations. We quantify this by the number of function evaluations or the total wall-clock GPU time required to reach a target RMSE threshold.
- Simple regret / cumulative regret: Derived from multi-armed bandit literature, simple regret measures the MSE gap between the globally optimal configuration and the agent's final recommended hyperparameter set. Cumulative regret measures the total performance penalty accumulated during the entire search phase.

3.2 Baseline Methods

We compare against the following standard HPO baselines:

- Random Search [Bergstra and Bengio, 2012]: A fundamental baseline that randomly samples hyperparameter configurations. It serves as a necessary lower bound to verify that the proposed RL agent explores high-dimensional continuous search spaces more effectively than random selection.
- BO [Snoek et al., 2012]: A standard state-of-the-art method for HPO. We will use a Gaussian Process-based BO with Expected Improvement (EI) to systematically benchmark against our RL agent's sequential decision-making capabilities.
- Hyp-rl [Jomaa et al., 2019]: An RL-based HPO method that learns a sequential decision policy via DQN over a discretized hyperparameter space, using only scalar reward feedback without learning curve information.

In addition, we evaluate two variants of our proposed method:

- **CrossDataset-HyperRL**: A Hyp-RL agent pre-trained across multiple source datasets and evaluated on held-out target datasets, isolating the contribution of cross-dataset policy transfer (idea ii).
- **CrossDataset-LC-DQN**: Our full method, which combines cross-dataset pre-training with the derivative-informed state representation (idea i), augmenting the agent's input with first- and second-order derivatives of the observed learning curves.

3.3 Benchmark Tasks or Datasets

We evaluate the algorithms in various benchmark domains widely used in the literature, specifically focusing on continuous prediction tasks:

- **LCBench** [Zimmer et al., 2021]: A learning curve benchmark built on top of OpenML datasets, providing pre-computed training trajectories of funnel-shaped MLPs across 35 tabular classification tasks. Each of its 2,000 hyperparameter configurations per dataset is logged with per-epoch validation losses over 50 epochs, exposing the full learning curve required by our derivative-informed state representation. The shared 7-dimensional search space (batch size, learning rate, momentum, weight decay, number of layers, max units per layer, dropout) and aligned training protocol across datasets make LCBench particularly well-suited for evaluating cross-dataset policy transfer.
- **NAS-Bench-360** [Tu et al., 2021]: While traditional NAS benchmarks focus on classification, NAS-Bench-360 extends to diverse domains including 1D sequence and continuous prediction. We plan to utilize this benchmark in future work to evaluate the RL agent's capability in navigating discrete structural hyperparameter spaces (e.g., layer types, channel sizes) for regression models.

4 Methodology

Our framework, illustrated in Figure 1, extends the Hyp-RL formulation [Jomaa et al., 2019] along two axes: a derivative-informed state representation and a cross-dataset meta-training procedure. Both share the same DQN policy and differ only in state features and training protocol.

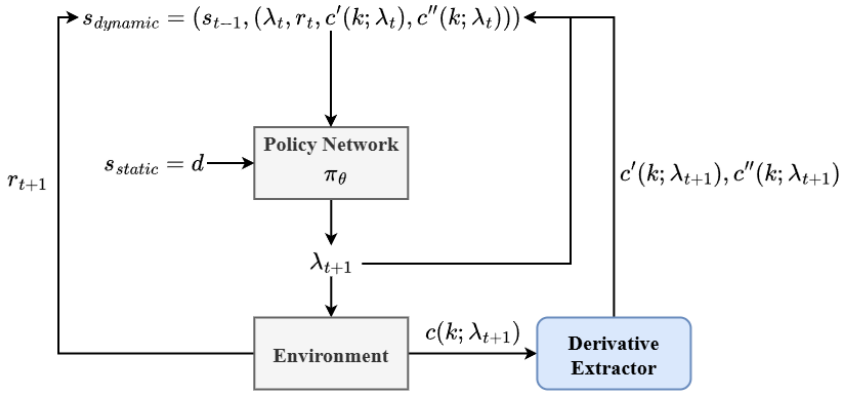


Figure 1: Framework

Derivative-informed state. Rather than feeding the full derivative sequence $c'(k)$, $c''(k)$ into the network, we summarize each observed learning curve with five compact finite-difference statistics over its last 8 epochs: the final value, minimum, mean first difference (slope), mean second difference (curvature), and total improvement. These features are appended to each history row, increasing the input dimension by five. We refer to this variant as **CrossDataset-LC-DQN**, with **CrossDataset-HyperRL** denoting the ablation without curve features.

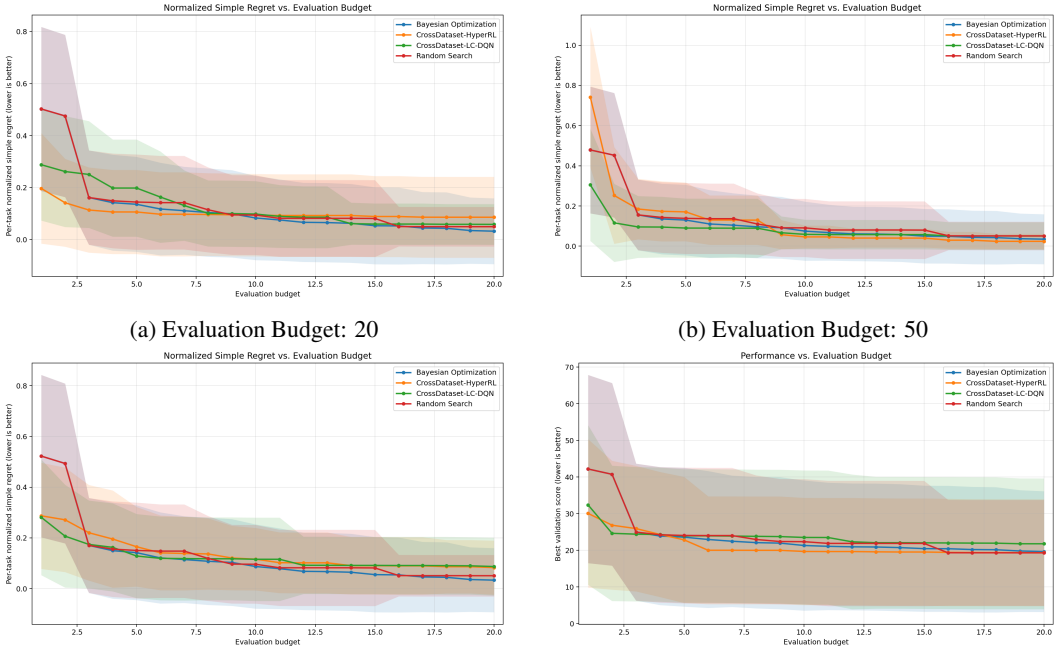
Cross-dataset meta-training. We learn a single shared DQN across source datasets via round-robin sampling: each meta-training episode samples one source dataset, runs a full HPO episode of length T, and updates the shared policy from a shared replay buffer. To stabilize learning across heterogeneous loss scales, we use an improvement reward $r_t = \max(0, -f_t + f_{t-1})$. Dataset meta-features (log-normalized counts of classes, features, and instances) are appended to every history row and, in the LSTM variant, also used to initialize the recurrent hidden state. After meta-training, the policy is frozen ($\varepsilon = 0.01$) and evaluated on held-out target datasets, while non-meta baselines are reinitialized per target under the same budget.

5 Experimental Results

5.1 Evaluation of Baseline Methods

Based on the experimental results evaluating validation scores against evaluation budgets, the following technical observations are made regarding the performance of the optimization algorithms:

- **Initial Optimization Phase (Cold-Start):** At low evaluation budgets (e.g., trials 1 through 5), the `CrossDataset-HyperRL` and `CrossDataset-LC-DQN` methods yield lower validation scores compared to Bayesian Optimization and Random Search. This indicates a higher search efficiency and a more effective initialization during the early phase of the optimization process.
- **Meta-Learning and Knowledge Transfer:** This early-stage performance difference correlates with the underlying initialization mechanisms. `CrossDataset-HyperRL` utilizes metadata to transfer learned hyperparameter distributions from prior datasets to the current validation dataset. Conversely, standard Bayesian Optimization initiates the search process on each new dataset without prior external data, requiring more exploratory trials to model the objective function from scratch.
- **Asymptotic Convergence:** As the evaluation budget increases (beyond 15 to 20 trials), the validation scores of all evaluated methods converge to a similar numerical range. The data shows that given a sufficient evaluation budget, the surrogate model constructed by Bayesian Optimization achieves an equivalent performance level to the transferred policies of the meta-learning approaches.



(a) Evaluation Budget: 20

(b) Evaluation Budget: 50

(c) Evaluation Budget: 100

(d) Evaluation Budget: 200

Figure 2: Performance comparison of hyperparameter optimization methods across different evaluation budget constraints. Lower validation scores indicate better performance.

6 Expected Contributions of Each Team Member

- Han-Syuan Liao: Focuses on result presentation, including data visualization, poster design, and delivering the final presentation.
- Cheng-An Tsai: Responsible for empirical evaluation, including dataset selection, experimental setup, and parameter tuning for model fine-tuning.
- Yi-Xiang Yu: Leads the problem formulation and methodology design, providing theoretical foundations and ensuring alignment with project goals.

References

Jeroen Albrechts, Hugo M. Martin, and Maryam Tavakol. Model-based meta-reinforcement learning for hyperparameter optimization. In *Intelligent Data Engineering and Automated Learning – IDEAL 2024*, volume 15346 of *Lecture Notes in Computer Science*. Springer, Cham, 2025.

James Bergstra and Yoshua Bengio. Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 2012.

Hadi S. Jomaa, Josif Grabocka, and Lars Schmidt-Thieme. Hyp-rl: Hyperparameter optimization by reinforcement learning. *arXiv preprint arXiv:1906.11527*, 2019.

Jasper Snoek, Hugo Larochelle, and Ryan P Adams. Practical bayesian optimization of machine learning algorithms. In *Advances in Neural Information Processing Systems*, 2012.

Ruochen Tu et al. Nas-bench-360: Benchmarking neural architecture search on diverse tasks. In *Advances in Neural Information Processing Systems*, 2021.

Martin Wistuba, Arlind Kadra, and Josif Grabocka. Supervising the multi-fidelity race of hyperparameter configurations. In *Advances in Neural Information Processing Systems*, volume 35, 2022.

Jia Wu, Xiyuan Liu, and Senpeng Chen. Hyperparameter optimization through context-based meta-reinforcement learning with task-aware representation. *Knowledge-Based Systems*, 260:110160, 2023.

Lucas Zimmer, Marius Lindauer, and Frank Hutter. Auto-PyTorch tabular: Multi-fidelity metalearning for efficient and robust AutoDL. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 43(9): 3079–3090, 2021.